
B28: SQL SuperHero Program : Session 33

1 message

Vishal Jaiswal <vishaldbdev@gmail.com>

13 April 2025 at 22:00

Bcc: satishhattekar1997@gmail.com

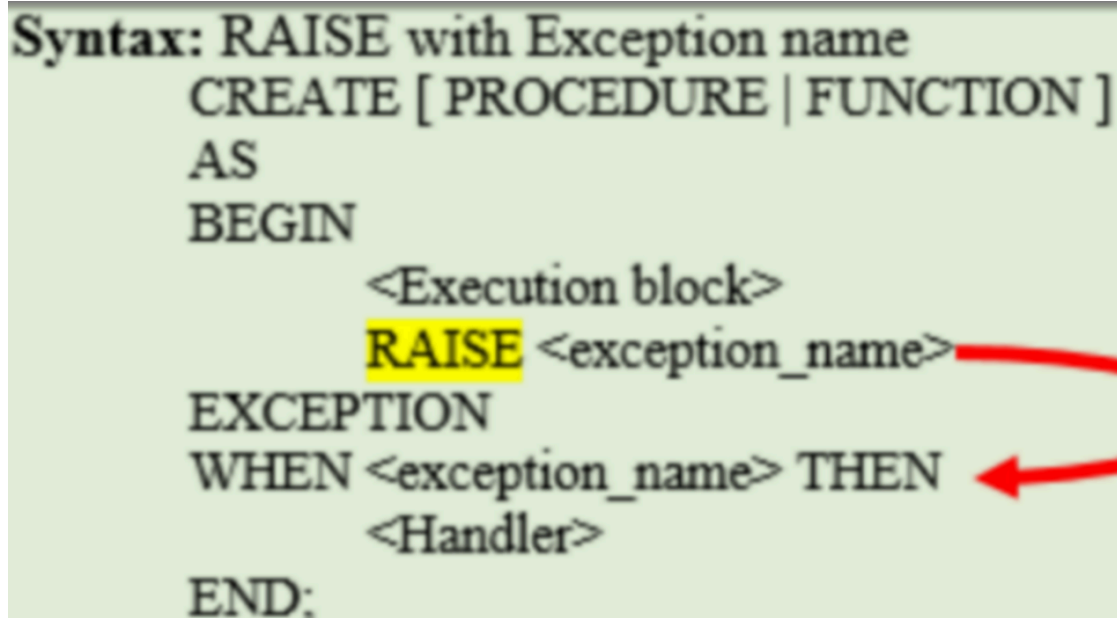
User Defined Exception

PL/SQL allows you to define your own exceptions according to the need of your program. the programmer can create their own exception and handle them. They can be created at a subprogram level in the declaration part. A user-defined exception must be declared and then raised explicitly, using either a RAISE statement or the procedure DBMS_STANDARD.RAISE_APPLICATION_ERROR.

The syntax for declaring an exception is –

DECLARE**my-exception EXCEPTION;**

- In the above syntax, the variable '**exception_name**' is defined as '**EXCEPTION**' type.
- This can be used as in a similar way as a predefined exception.
- All the predefined exceptions are raised implicitly whenever the error occurs. But the user-defined exceptions needs to be raised explicitly. This can be achieved using the keyword '**RAISE**'. This can be used in any of the ways mentioned below.



```
Syntax: RAISE with Exception name
CREATE [ PROCEDURE | FUNCTION ]
AS
BEGIN
    <Execution block>
    RAISE <exception_name>
EXCEPTION
WHEN <exception_name> THEN
    <Handler>
END;
```

```

declare
    vishal_exception exception;
Begin
    raise vishal_exception;
Exception
    when vishal_exception then
        dbms_output.put_line('my exception');
End;

```

✓ Defined

→ Raised

→ Handled

Statement processed.
my exception → output

=====

```

create table location (locid varchar2(20), minqty number, maxqty number);

```

I have a table called LOCATION, and I am writing a procedure to insert data in that table.

```

create or replace procedure insert_loc_data(p_locid varchar2, p_minqty number, p_maxqty number) as
err_loc_length exception;
err_mingmax exception;
err_invaliddata exception;
Begin
    If length(p_locid) > 5 then
        Raise err_loc_length;
    elsif p_minqty > p_maxqty then
        raise err_mingmax;
    elsif p_minqty > 999 or p_maxqty > 999 then
        raise err_invaliddata;
    else
        insert into location values (p_locid, p_minqty, p_maxqty);
    end if;

Exception
    When err_loc_length then
        raise_application_error(-20123, 'location code must be maximum 5 character long');
    When err_mingmax then
        raise_application_error(-20786, 'max qty should always more than min qty');
    When err_invaliddata then
        raise_application_error(-20458, 'your qty must be less than 1000');
End;

```

```

create or replace procedure insert_loc_data(p_locid varchar2, p_minqty number, p_maxqty number) as
err_loc_length exception;
err_mingmax exception;
err_invaliddata exception;
Begin
  If length(p_locid) > 5 then
    Raise err_loc_length;
  elsif p_minqty > p_maxqty then
    raise err_mingmax;
  elsif p_minqty > 999 or p_maxqty > 999 then
    raise err_invaliddata;
  else
    insert into location values (p_locid, p_minqty, p_maxqty);
  end if;
Exception
  When err_loc_length then
    raise_application_error(-20123,'location code must be maximum 5 character long');
  When err_mingmax then
    raise_application_error(-20786,'max qty should always more than min qty');
  When err_invaliddata then
    raise_application_error(-20458,'your qty must be less than 1000');
End;

```

Handwritten notes:

- Exception:** Indicated by a bracket on the exception declarations.
- Parameter:** Indicated by a bracket on the procedure parameters.
- Suppose someone try to insert more than 5:** An arrow points from this text to the condition `length(p_locid) > 5`.
- This is raised:** An arrow points from this text to the `Raise err_loc_length;` statement.
- error no.:** An arrow points from this text to the error code `-20123`.
- error message:** An arrow points from this text to the error message `'location code must be maximum 5 character long'`.

In the above example, I am trying to declare user defined exception depends on developer point.

If someone tries to insert a record in the table, it will check the rule , defined by the developer and if it is violated , raise the user defined exception here.

```

execute insert_loc_data('ABC123', 700, 900)

```

Handwritten notes:

- more than 5 character:** An arrow points from this text to the string `'ABC123'`.
- error code what we decided:** An arrow points from this text to the error code `ORA-20123`.
- error message what we decided:** An arrow points from this text to the error message `location code must be maximum 5 character long`.

ORA-20123: location code must be maximum 5 character long ORA-06512: at
ORA-06512: at line 1
ORA-06512: at "SYS.DBMS_SQL", line 1721

More Details: <https://docs.oracle.com/error-help/db/ora-20123>

I tried to run the above procedure and inserting the location code that is 'ABC123', length = 6 in the code, it was mentioned that:

```

if LENGTH(PLOCID)> 5 Then
  raise err_loc_length;

```

So, it raised this user defined exception. and go to exception block :

```

EXCEPTION
WHEN err_loc_length THEN
RAISE_APPLICATION_ERROR( -20123,'location code must be maximum 5 character long' );

```

The image shows two screenshots of Oracle error messages with handwritten annotations in purple ink.

Top Screenshot:

```
execute insert_loc_data('ABC12', 700, 600)
```

Annotations: "min" with an arrow pointing to 700, "max" with an arrow pointing to 600, and "min > max" written to the right.

Error messages:

```
ORA-20786: max qty should always more than min qty ORA-06512: at "SQL_OKCSTNDMCFCNZP"
ORA-06512: at line 1
ORA-06512: at "SYS.DBMS_SQL", line 1721
```

Annotation: "error message" with an arrow pointing to the error text.

More Details: <https://docs.oracle.com/error-help/db/ora-20786>

Bottom Screenshot:

```
execute insert_loc_data('ABC12', 700, 1100)
```

Annotation: "max > 1000" with an arrow pointing to 1100.

Error messages:

```
ORA-20458: your qty must be less than 1000 ORA-06512: at "SQL_OKCSTNDMCFCNZPQWVCMQ5J"
ORA-06512: at line 1
ORA-06512: at "SYS.DBMS_SQL", line 1721
```

Annotations: "error no." with an arrow pointing to the error number, and "msg what we want to display" with an arrow pointing to the error message.

More Details: <https://docs.oracle.com/error-help/db/ora-20458>

Raise Application Error

The `raise_application_error` is actually a procedure defined by Oracle that allows the developer to raise an exception and associate an error number and message with the procedure. This allows the application to raise application errors rather than just Oracle errors. **Error numbers are defined between -20,000 and -20,999.**

--> If I declare `Raise_Application_Error` with error numbers not in 20000 and 20999, it will throw error that out of range. All other numbers belong to Oracle for its own errors.

The image shows a screenshot of an Oracle error message with a handwritten annotation.

Page number: 36

Error message:

```
ORA-21000: error number argument to raise_application_error of -21234 is out of range
ORA-06512: at line 2
ORA-06512: at "SYS.DBMS_SQL", line 1721
```

Annotation: "when other than range" with an arrow pointing to the error number -21234.

SAME ERROR NUMBER FOR ALL ERROR

Exception

```
when err_loc_length then
    raise_application_error(-20134, 'location code must be mix 5 character long');
when err_mingmax then
    raise_application_error(-20134, 'max qty should always be greater thah min qty');
when err_invaliddata then
    raise_application_error(-20134, 'your qty must be between 0 and 999');
```

End;

Procedure created.

Same no. for different exception

✓ yes, allowed

See above, yes if we have many user defined exceptions , all may have the same numbers. There is no issue. But it is not advisable, because it creates issues while debugging the code that problem is coming from which place.

Can I give error number any thing other than between -20000 and -20999

Exception

```
when err_loc_length then
    raise_application_error(-20134, 'location code must be mix 5 character long');
when err_mingmax then
    raise_application_error(-20134, 'max qty should always be greater thah min qty');
when err_invaliddata then
    raise_application_error(-12345, 'your qty must be between 0 and 999');
```

End;

Procedure created.

out of range (range allowed between -20000, -20999)

→ NO compilation error

See above code, I have given the error number as -12345 (out of range, range is -20000 to -20999) but still no issue in compilation. It only creates issues at run time and when this exception comes. Suppose if any other exception comes and the compiler enters into the exception block, but not in err_pmingmax, then still there is no issue.

```
raise_application_error(-20134, 'please enter correct value , maximum qty is out of range');
When err_pmingmax then
    raise_application_error(-22234, 'minimum is always less than maximum');
When Others Then
    raise_application_error(-20000, sqlerrm);
End add_location_to_db;
```

Procedure created.

```
EXEC add_location_to_db('Loc12',500,1400)
```

ORA-20134: please enter correct value , maximum qty is out of range ORA-06512: at "SQL_

ORA-06512: at line 1

ORA-06512: at "SYS.DBMS_SQL", line 1721

it only gives error (out of range) if that exception occurs, not for any other exception, like in above example, we have one exception defined with number 22234, which is out of range, but no issue at all, as here it comes another error.

But it comes the out of range error, when that exception comes at run time, only if that particular error come:

```
execute insert_loc_data('ABC15', 700, 1800)
```

ORA-21000: error number argument to raise_application_error of -12345 is out of range

ORA-06512: at line 1

ORA-06512: at "SYS.DBMS_SQL", line 1721

Can We do System defined and User defined both together?

YES

Exception

when no_data_found then

null;

when err_loc_length then

raise_application_error(-20134, 'location code must be mix 5 character long');

when err_mingmax then

raise_application_error(-20134, 'max qty should always be greater thah min qty');

when err_invaliddata then

raise_application_error(-12345, 'your qty must be between 0 and 999');

End;

Procedure created.

→ System defined

→ user defined

→ Yes

=====

Raise Application Error Use in a Function :

Remember one point, UDE is just a program, a facility that checks user activity and if we want to stop user and give a meaningful message, we can use UDE.

create or replace function get_sal(p_empid number) return number as

l_salary number;

Guru exception;

Begin

if p_empid = 100 then

raise Guru;

Else

select salary into l_salary from hr.employees where employee_id = p_empid;

End If;

Return l_salary;

Exception

when Guru then

raise_application_error (-20333, 'you are not allowed to checkj this salary buddy, seriously!!!! can in report yiou');

End;

```

create or replace function get_sal(p_empid number) return number as
l_sal number;
guru exception; user defined
Begin
  If p_empid = 100 then
    raise guru; raised
  else
    select salary into l_sal from hr.employees where employee_id = p_empid;
  end If;
  return l_sal;
Exception
  when guru then Handled
    raise_application_error(-20333, 'you are not allowed to check this salary buddy !! can i report you');
End;

Function created.

```

✓ good here

```

select get_sal(112) from dual

```

GET_SAL(112)
7800

issue here

```

select get_sal(100) from dual

```

ORA-20333: you are not allowed to checkj this salary buddy, seriously!!!! can in report yiou

See the above example, There is a function, which gives you a salary. I have added a condition that it is not allowed for emp_id = 100. Raise shanti. What is shanti??? an exception, defined in the declare section.

=====

Why are we using Raise_Application_Error, if we can give the same message via DBMS_OUTPUT.PUT_LINE()

RAISE_APPLICATION_ERROR and DBMS_OUTPUT.PUT_LINE lies in their purpose and functionality within a PL/SQL program.

RAISE_APPLICATION_ERROR:

- Used to explicitly raise a user-defined error in a PL/SQL block.
- It Aborts the current transaction when invoked.
- It is for Transaction control. If error ocured, it stop and compiler goes to Exception.
- Returns a custom error code and message to the application.
- We use for implementing business logic.

DBMS_OUTPUT.PUT_LINE:

- Simply outputs a message to the standard output buffer.
- Primarily used for debugging or logging purposes.
- Does not raise an error or halt execution of the program.
- The program continues execution as if nothing went wrong.

Interview question

```

declare
  l_count number;
Begin
select count(*) into l_count from hr.employees where department_id = 900;
dbms_output.put_line('Hi');
Exception
  when no_data_found then
    dbms_output.put_line('Hello');
End;

```

In the above code, Interviewer will tell you that there is no data for department_id = 900. So, The program will throw NO_DATA_FOUND exception or not?

NO. It will print HI. because if there is no data for department_id = 900 so count(*) = 0. it means the variable l_count = 0.

NO_DATA_FOUND will come when null, or no data. but here we get data 0.

remember, count(*) will never throw NO_DATA_FOUND exception.

Pragma Exception INIT

PRAGMA is not executable code, instead, a PRAGMA is an instruction to the PL/SQL compiler.

The exception_init PRAGMA instructs Oracle to assign a name to a standard Oracle error message that does not have an associated named exception.

Pragma is a keyword directive to execute proceed at compile time. pragma EXCEPTION_INIT function take this two argument,

- exception_name
- error_number

You can define pragma EXCEPTION_INIT in DECLARE BLOCK on your program.

```

1 DECLARE
2   l_count number;
3   many_records EXCEPTION;
4   PRAGMA EXCEPTION_INIT(many_records, -20222);
5 BEGIN
6   Select count(*) into l_count from hr.employees where department_id = 90;
7   IF l_count < 10 THEN
8     RAISE many_records;
9   END IF;
10 EXCEPTION
11   WHEN many_records THEN
12     dbms_output.put_line('I tested pragma exception');
13 END;

```

Handwritten annotations in red:

- An arrow points from "Name of exception" to `many_records` in line 3.
- An arrow points from "Number" to `-20222` in line 4.
- An arrow points from "exception" to the `EXCEPTION` block in line 10.

Output at the bottom:

```

Statement processed.
I tested pragma exception

```

Same program in Raise_Application_Error:

Declare l_count Number; many_records exception; Begin select count(*) into l_count from hr.employees where employee_id = 90; If l_count < 10 then

```
raise many_records; End If; Exception When many_records Then raise_application_error(-20345, 'there are very less records'); End;
```

```
Declare
```

```
l_count Number;
```

```
many_records exception;
```

```
Begin
```

```
select count(*) into l_count from hr.employees where employee_id = 90;
```

```
If l_count < 10 then
```

```
raise many_records;
```

```
End If;
```

```
Exception
```

```
When many_records Then
```

```
raise_application_error(-20345, 'there are very less records');
```

```
End;
```

```
ORA-20345: there are very less records ORA-06512: at line 11
```

```
ORA-06512: at "SYS.DBMS_SQL", line 1721
```

Difference Between Raise Application Error And Pragma Exception INIT

PRAGMA EXCEPTION_INIT allows to associate an Oracle error number with the name of a user-defined error. Here you need to defined user-defined exception before it is used in PRAGMA EXCEPTION_INIT.

There are two parameters: exception name and error code.

RAISE_APPLICATION_ERROR allows to create meaningful error [msg.it](#) works with un-named user defined exceptions. It associates the number of the error with the text of the error. Therefore, the user-defined exception does not have a name associated with it.

There are two uses for RAISE_APPLICATION_ERROR. The first is to replace generic Oracle exception messages with our own, more meaningful messages. The second is to create exception conditions of our own, when Oracle would not throw them

Explain Pragma Exception Init

Remember Pragma Exception init is used to give a name to oracle standard message (less than 20000).

Raise Application error is used to give message to user defined error.

Example:

```
CREATE TABLE supplier
```

```
( supplier_id number(10) primary key,
```

```
supplier_name varchar2(50) not null,
```

```
contact_name varchar2(50)
```

```
);
```

```
CREATE TABLE products
```

```
( product_id number(10) not null,
```

```
supplier_id number(10) not null
```

```
);
```

```
insert into supplier values (1, 'azx','yryryr');
```

```
insert into supplier values (2, 'rrr','hyuj');
```

```
insert into supplier values (3, 'ggg','seedf');
```

```
insert into supplier values (4, 'uuu','ghyujk');
```

```
insert into products values (1,1);
```

```
insert into products values (1,4);
```

```
ALTER TABLE products
```

```
ADD CONSTRAINT fk_vishal
```

FOREIGN KEY (supplier_id)
REFERENCES supplier(supplier_id);

We created tables with parent child relationships. If we try to delete data from supplier, it will not allow because it gives error as child found. This is oracle standard error message and it will give error

```
delete from supplier where supplier_id = 1
```

ORA-02292: integrity constraint (SQL_MOAXWMEGAZRNGIFLBNCJRFPM.FK_VISHAL) violated - child record found

```
Declare
  v_supp_id number := 1;
Begin
  delete from supplier where supplier_id = v_supp_id;
End;
```

ORA-02292: integrity constraint (SQL_MOAXWMEGAZRNGIFLBNCJRFPM.FK_VISHAL) violated - child record found
ORA-06512: at "SYS.DBMS_SQL", line 1721

standard error

Lets use Pragma Exception INIT:

```
Declare
  v_supp_id number := 1;
  child_found exception;
  pragma exception_init(child_found, -02292);
Begin
  delete from supplier where supplier_id = v_supp_id;
Exception
  when child_found then
    dbms_output.put_line('this is not allowed, child data existss. yopu can not do this');
End;
```

```
1
2
3 declare
4 v_supp_id number := 1;
5 child_found exception;
6 pragma exception_init(child_found, -02292);
7 Begin
8     delete from supplier where supplier_id = v_supp_id;
9 Exception
10 when child_found then
11     dbms_output.put_line('This is not allowed, child data excists');
12 End;
13
```

Statement processed.
This is not allowed, child data excists

Handwritten notes:
- Red arrow from line 6 to "we gave a name"
- Blue arrow from "oracle standard error" to line 6
- Red arrow from the output message to "our message displayed"

--> If you see the above code, Pragma Exception_INIT will adopt the oracle standard error and display a meaningful message.

```
declare
    v1 number := 10;
    v2 number := 0;
    v3 number;
Begin
    v3 := v1/v2;
End;
```

Handwritten notes:
- Red arrow from "standard error" to the error message
- Green arrow from "Standard message" to the error message

ORA-01476: divisor is equal to zero ORA-06512: at line 6
ORA-06512: at "SYS.DBMS_SQL", line 1721

More Details: <https://docs.oracle.com/error-help/db/ora-01476>

Handwritten note:
- Green arrow pointing down: "Let's adopt it via pragma below"

Name 9
wrong

```
declare
    v1 number := 10;
    v2 number := 0;
    v3 number;
    do_not_do_it exception;
    pragma exception_init(do_not_do_it, -01476);

Begin
    v3 := v1/v2;
exception
    when do_not_do_it then
        dbms_output.put_line('please man, you are doing wrong');
End;
```

Statement processed.
please man, you are doing wrong

→ This was the error, we want to adopt

→ we called

→ my message

-- You can see here that Pragma Exception_INIT help you to adopt any oracle standard message. But there is a special case for NO DATA FOUND

```
declare
    l_name varchar2(20);

Begin
    select first_name into l_name from hr.employees where salary = 17000;
    dbms_output.put_line(l_name);

End;
```

→ This is standard error TOO-MANY-ROWS

ORA-01422: exact fetch returns more than requested number of rows ORA-06512: at line 4
ORA-06512: at "SYS.DBMS_SQL", line 1721

More Details: <https://docs.oracle.com/error-help/db/ora-01422>

```

declare
    l_name varchar2(20);
Begin
    select first_name into l_name from hr.employees where salary = 17000;
    dbms_output.put_line(l_name);
exception
    when too_many_rows then
        dbms_output.put_line('so many');
End;

Statement processed.
so many

```

→ let's adopt it

```

declare
    l_name varchar2(20);
    sachin exception;
    pragma exception_init(sachin,-01422);
Begin
    select first_name into l_name from hr.employees where salary = 17000;
    dbms_output.put_line(l_name);
exception
    when sachin then
        dbms_output.put_line('so many');
End;

Statement processed.
so many

```

we called it Sachin
→ this was the error

→ called

our message

If we adopt the exception and also raise the same exception, it will compile time issue. here Sachin means 01422 and same too_many_rows, so database got confused and says compile issue

```

declare
    l_name hr.employees.first_name%type;
sachin exception;
pragma exception_init(sachin,-01422);
Begin
select first_name into l_name from hr.employees where salary = 17000;
exception
    when sachin then
        dbms_output.put_line('Hi');
    when too_many_rows then
        dbms_output.put_line('Hello');
End;

```

Both are same

ORA-06550: line 8, column 6:

PLS-00484: redundant exceptions 'TOO_MANY_ROWS' and 'SACHIN' must appear in same exception handler

=====

```

declare
my_exception exception;
pragma exception_init(my_exception,100);
Begin
    raise my_exception;
Exception
    when my_exception then
        dbms_output.put_line('there is no data');
End;

```

this is no-data-found

adopted as my-Exception

basically no-data-found

Handled

Statement processed.
there is no data